

World Cup Analytics Capstone

Step 2 — SQL Transformation Guide: turning raw data into a real model

1. Where we left off

By the end of `0_setup_guide.pdf`, you ran two Python scripts and ended up with **33 raw tables** sitting in your Supabase database — 32 downloaded straight from GitHub, plus 1 small hand-verified table of 2026 award winners. Every one of those tables is prefixed `raw_`, and every one of them is still messy: the two GitHub sources don't agree on team names, one of them mixes men's and women's tournaments together, and neither is shaped like something you could build a report on directly.

Nothing about that is a mistake — it's the plan working as intended. Landing data raw first, then cleaning it afterward, is what lets every cleaning decision from here on be visible and reversible, instead of buried invisibly inside a download step.

2. The story: what's actually wrong with this data

Before opening any of the three SQL files, it's worth understanding what they're actually for. These two GitHub datasets were built independently, by different people, years apart, for different purposes. Neither was designed to be joined with the other. That's exactly why combining them is interesting — and exactly why it takes real, deliberate work rather than a single button press.

No relationships at all

Every one of the 33 raw tables is flat — a plain spreadsheet with no primary keys and no foreign keys connecting it to any other table. A tool like Power BI can't filter a match by its team, or a goal by its match, until something establishes those connections explicitly.

Two ID systems that don't speak to each other

The historical source identifies teams with text codes like "T-01". The 2026 source uses plain integers like 1. These aren't compatible in any way — there's no formula to convert one into the other. The only reliable way to connect a 2026 team to its historical record is by matching its *name* instead of its ID.

The same country, two different names

Which immediately creates a second problem: the two sources don't always agree on what to call a country. The 2026 data says "Türkiye"; the historical data says "Turkey". Same with Iran, Ivory Coast, Czech Republic, and the United States. Left alone, a name-based match would treat these as eight different countries instead of four.

Two competitions mixed into one

The historical file contains both the men's and women's World Cups in a single table, distinguished only by a text label. This project is scoped to men's tournaments only — so a real decision has to be made about where that line gets drawn, and it has to be drawn consistently everywhere: matches, goals, cards, and awards alike.

A tournament that doesn't exist yet

The historical dataset was published before the 2026 tournament happened, so it simply stops at 2022. There is no row anywhere in it representing "the 2026 World Cup" — which means nothing can properly reference it until one gets created by hand.

Time written as text, not numbers

The 2026 event data records stoppage-time goals as text, like "90+3", instead of a plain number. Any attempt to do arithmetic on that column — average goal timing, for instance — fails immediately unless that text gets interpreted correctly first.

Everything mixed into one big list of events

The 2026 dataset logs goals, yellow cards, red cards, assists, and VAR reviews all in one shared table, distinguished only by a label. Counting "goals" without carefully filtering that label first would silently also count cards and reviews.

Rich, but with nowhere to put it

The 2026 source also includes things no historical tournament ever recorded: expected goals (xG), player market value, ELO ratings, possession, referee statistics. These are genuinely valuable, but they can't simply be added as new columns onto the historical tables — doing so would leave 92 years of empty values and invite unfair "highest xG ever" style comparisons that only look at one data point.

Bare numbers with no names attached

Inside the 2026 match data, the referee and "player of the match" are stored as plain ID numbers, pointing at nothing. There's no table yet that turns those numbers into an actual name.

Two gaps that need different fixes

Finally, two smaller but real gaps: no dataset anywhere records who won the 2026 awards (Golden Boot, Golden Ball, and so on) — that has to be added by hand, verified against real sources. And cards are recorded completely differently per source: a dedicated table historically, buried inside the general event log for 2026 — two different shapes that need two different extraction methods to end up in the same place.

Same country, different era — our call to make

History doesn't always draw clean lines around "one team." West and East Germany reunified in 1990. Czechoslovakia split into the Czech Republic and Slovakia in 1993. The Soviet Union and Yugoslavia each dissolved into several present-day countries. Even FIFA isn't fully consistent about these — it merges West Germany's record into Germany's, but keeps East Germany separate, for instance. Rather than quietly inherit FIFA's mixed approach, this project makes its own explicit decisions: Germany's two halves are merged into one team (deliberately simpler than FIFA's own rule). Zaire is merged into DR Congo, since it's a straight rename of the same country, not a split. Czechoslovakia, the Soviet Union, and Yugoslavia all stay as their own separate, independent teams — every successor state (Czech Republic, Slovakia, Russia, Ukraine, Serbia, Croatia, and the rest) starts fresh with zero inherited history. None of this happens automatically or invisibly — every one of these decisions lives as a plain, readable row in `team_name_overrides`, so it's a choice you can point to and defend, not a rule buried in code.

A text column that didn't get the memo

One more real bug worth knowing, because the lesson generalizes well beyond this project: `dim_tournament` stores its winner as plain text, copied straight from the historical source. When West Germany and Zaire were merged into Germany and DR Congo, that text column wasn't automatically updated — it still said "West Germany" for 1954, 1974, and 1990, long after `dim_team` stopped having a row by that name. Any measure comparing a tournament's winner against a team's name would have silently missed those titles — no error, just a quietly wrong count. The fix is a second pass: an `UPDATE` statement that runs after `team_name_overrides` already exists, re-mapping the winner text through the same overrides used everywhere else. The general lesson: **any column that duplicates a name elsewhere in your model has to be kept in sync by hand whenever that name changes** — it never updates itself just because the "real" table did.

3. The objective: what these three scripts will achieve

Every problem above gets solved, once, by three SQL scripts run in order. By the end, the 33 messy, disconnected raw tables become **15 clean, properly connected tables**:

- One team list, one tournament list, one venue list, one stage list — each country, tournament, and stadium appearing exactly once, correctly linked across both eras
- One continuous timeline of matches and goals, 1930 through 2026, filtered consistently to men's tournaments only
- A separate, richer layer holding everything unique to 2026 — xG, market value, ELO, possession, referees — connected back to the same shared team and venue lists, without forcing that detail onto history
- Unified awards and cards spanning the full 96 years, with the one real data gap (2026 awards) filled by hand, verified information

That's the goal to keep in mind while reading the three files below — each one is solving a specific piece of the problem list above, not just "doing SQL stuff." All of this happens in **SQL**, not Python, and entirely **inside** your Supabase database — nothing runs on your laptop this time. You've kept these three files in their own subfolder, separate from the Python scripts — a sensible way to organize it, since these are all run the same way (pasted into Supabase's SQL Editor), not from Command Prompt like the Python scripts were.

4. The three files, and what each one solves

File	Builds	Solves
<code>1_transform_star_schema.sql</code>	6 core tables: <code>dim_team</code> , <code>dim_tournament</code> , <code>dim_venue</code> , <code>dim_stage</code> , <code>fact_matches</code> , <code>fact_goals</code>	Reconciles team names, filters to men's-only, adds 2026 to the tournament list, fixes broken minute formatting
<code>2_transform_2026_deepdive.sql</code>	7 tables: 2026 team/venue/referee/player detail, match stats, full event log	Adds rich 2026-only detail (xG, market value, ELO, possession) without forcing it onto historical data

3_transform_awards_bookings.sql	fact_awards, fact_bookings	Fills the 2026 awards gap; unifies yellow/red cards across both eras
---------------------------------	----------------------------	--

Each file also has its own detailed problem list written as comments at the very top of the script itself — open any one of them and read the first 15 lines before running it, to see exactly what it's about to fix and why.

5. Why the order matters

These numbers aren't arbitrary — each script needs tables the previous one built. File 2 looks for `dim_team` and `fact_matches`, which only exist after File 1 runs. File 3 looks for `fact_match_events_2026`, which only exists after File 2 runs. Running them out of order will fail with a "relation does not exist" error — if you ever see that error, it almost always means a numbered step got skipped.

6. The procedure — repeat for each of the 3 files, in order

- 1 Open your Supabase project → **SQL Editor** → click + for a new query.
- 2 Open the .sql file on your computer, select all the text, copy it.
- 3 Paste the whole thing into the query editor.
- 4 Rename the query (top of the tab) to match the filename — makes it easy to find again.
- 5 Click **Run**.
- 6 Run the sanity-check queries listed in a comment block at the bottom of that same file, one at a time, and compare what you get against the expected values below.

"Success. No rows returned" is normal, not an error — these scripts are all CREATE TABLE and INSERT statements, not SELECTs, so there's nothing to display until you run the separate sanity-check queries yourself.

7. What to check after each file

After running...	Run this check	Expect
1_transform_star_schema.sql	SELECT source, COUNT(*) FROM fact_matches GROUP BY source;	~964 (jfelstul) 104 (mominullptr_2026)
	SELECT COUNT(*) FROM fact_matches WHERE home_team_key IS NULL OR away_team_key IS NULL;	0
	SELECT team_name FROM dim_team WHERE team_name IN ('Germany', 'Congo DR', 'West Germany', 'Zaire');	Germany, Congo DR only (no West Germany/Zaire)

	SELECT DISTINCT winner FROM dim_tournament WHERE winner ILIKE '%germany%';	Germany only
2_transform_2026_deepdive.sql	SELECT COUNT(*) FROM dim_player_2026;	1248
3_transform_awards_bookings.sql	SELECT source, COUNT(*) FROM fact_awards GROUP BY source;	manual_2026 = 6

If a number doesn't match, don't move on to the next file — fix it first. Each script is safe to re-run as many times as needed; it drops and rebuilds its own tables every time.

8. Final checkpoint before moving on

Once all three files run cleanly and every check above matches, open Supabase's **Table Editor** and confirm you now see **15 new tables** without the raw_ prefix, alongside your original 33 raw ones:

- dim_team, dim_tournament, dim_venue, dim_stage, fact_matches, fact_goals
- dim_team_2026_attributes, dim_venue_2026_attributes, dim_referee_2026, dim_player_2026, fact_matches_2026_detail, fact_match_team_stats_2026, fact_match_events_2026
- fact_awards, fact_bookings

This is the real finish line for the database side of the project. Every later step (Power BI, DAX, the published report) only ever reads from these 15 tables — never from the raw ones again.

The next stage connects Power BI Desktop to this database and starts building the actual report — a separate guide, for a separate day.